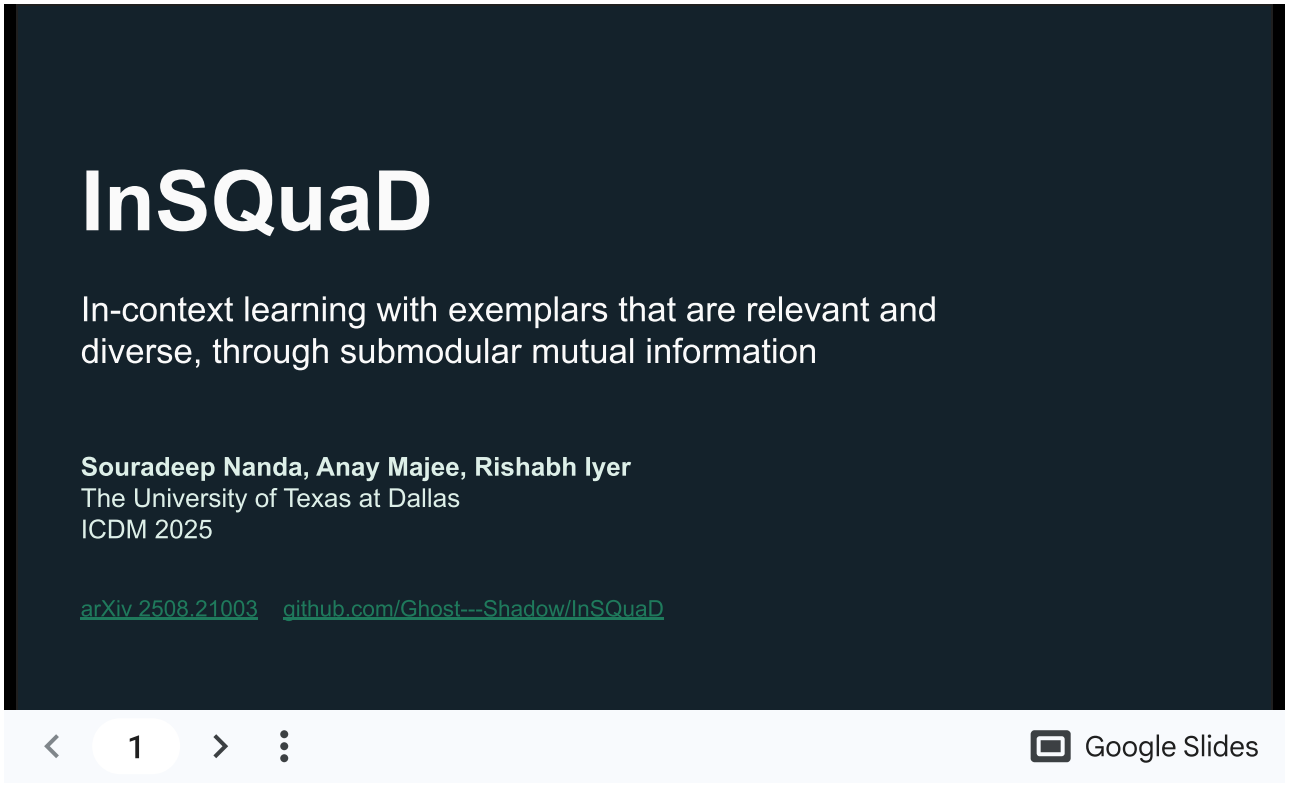
InSQuaD: Picking In-Context Examples That Are Relevant and Diverse

10 Nov 2026

A plain-language walk through our ICDM 2025 paper. One submodular function picks the examples to label, picks the examples for each prompt, and trains the retriever.

InSQuaD is a paper by Anay Majee, Rishabh Iyer and me at The University of Texas at Dallas. It was published at ICDM 2025. This post explains it for a reader who knows what an LLM prompt is, but not what submodularity is. The slides are public. The paper is on arXiv (the arXiv version is the long version of the ICDM paper) and the code is on GitHub.



The short version: we pick in-context examples with one function that rewards relevance to the query and penalises redundancy. We use the same function to choose what to annotate, to choose what to put in each prompt, and to train the retriever. On nine benchmarks with Gemma 2B, it beats eight baselines on average score.

The problem

In-context learning (ICL) means that an LLM gets a few worked examples, called exemplars, in the prompt. Then it gets the test query. Which exemplars go in the prompt decides how well this works.

There are two selection problems:

1. Annotation. We have a large pool of unlabeled examples. We choose a small set to label, within a budget B. Labels cost money.
2. Retrieval. For each test query, we choose k labeled exemplars. The order of the exemplars in the prompt also matters.

Each simple rule fails in its own way. Relevance alone returns near-duplicates: several examples that say the same thing. Diversity alone covers the pool, but drifts away from the query. A good prompt needs exemplars that are near the query and different from each other.



This figure is an illustration with synthetic points. It is not data from the paper. The star is the test query and the grey dots are candidate examples. The coloured dots are the four that each rule selects.

The paper names four limits of earlier work:

- Selective annotation methods, for example Vote-K and IDEAL, focus on relevance. They do not model diversity together with it.
- Pretrained sentence embeddings, for example SBERT, favour similarity to the query. Paraphrases and distractors still score high.
- Multi-stage pipelines treat relevance and diversity in separate stages.
- Multi-hop QA training sets have no paraphrases, so a retriever does not learn to skip duplicates.

One function for relevance and diversity

The tool we use is Submodular Mutual Information (SMI). Write it as I_f(A; Q). It is the information that a set of exemplars A shares with a query set Q.

A submodular function has diminishing returns. A second copy of the same example adds little to the set. So one function rewards relevance to Q and also penalises redundancy inside A. We do not need two stages.

We use three instantiations. S is a similarity kernel between two items, and eta is a weighting term.

Name	Formula	In words
Graph-Cut (GC)	sum over a in A, b in Q of S(a, b)	Total similarity between A and Q.
Facility-Location (FL)	sum over a in A of max over b in Q of S(a, b), plus eta times sum over b in Q of max over a in A of S(a, b)	Each side is covered by its best match.
Log-Determinant (LD)	log det(S_A) - log det(S_A - eta^2 * S_AQ * S_Q^-1 * S_AQ^T)	The volume of A that Q explains.

InSQuaD-RETRIEVE

InSQuaD-RETRIEVE uses the same greedy rule in two stages.

1. Annotate, once. Pick at most B examples from the pool V that maximise I_f(shortlist; V). Here the "query" is the full pool, so this picks a set that represents the pool. People then label the shortlist.
2. Retrieve, for each test query q. Pick at most k labeled examples C that maximise I_f(C; q).

The greedy rule is simple. Start with an empty set. Add the example with the largest gain in I_f. Repeat. The order of selection is the order of decreasing gain, and we use that order as the order in the prompt.

InSQuaD-LEARN

The selection rule needs a similarity S. A pretrained embedding model gives one, but it was not trained for this. InSQuaD-LEARN trains the retrieval (embedding) model so that its similarities suit the same function.

For each training question q there are three sets:

- S+, the positives (relevant documents).
- S-, the negatives (distractors).
- Sp, the paraphrases of the positives.

The quality loss raises the overlap of the query with the positives and lowers it with the negatives:

```
L_q = sum [ log I_f(S-; q) - log I_f(S+; q) ]
```

The diversity loss pulls each positive and its paraphrase together, so that a paraphrase is not retrieved as a new exemplar:

```
L_d = sum [ log I_f(S-; Sp) - log I_f(S+; Sp) ]
```

The joint loss, as written in the paper, mixes the two with a weight lambda in [0, 1]:

```
L = exp((1 - lambda) * L_q + lambda * L_d)
```

With lambda = 0 the loss uses quality only. The loss is a likelihood ratio from a submodular point process. Only the retriever is trained. The LLM is not changed.

The training data is HotpotQA. It gives supporting documents and distractor documents for each question. We added one paraphrase for each document, written by GPT-3.5 Turbo. The retriever is an SBERT model with MPNet. We trained it for 7 epochs with batch size 32, learning rate 3e-5, AdamW and weight decay 0.01. This took about 527 L4 GPU hours.

Setup

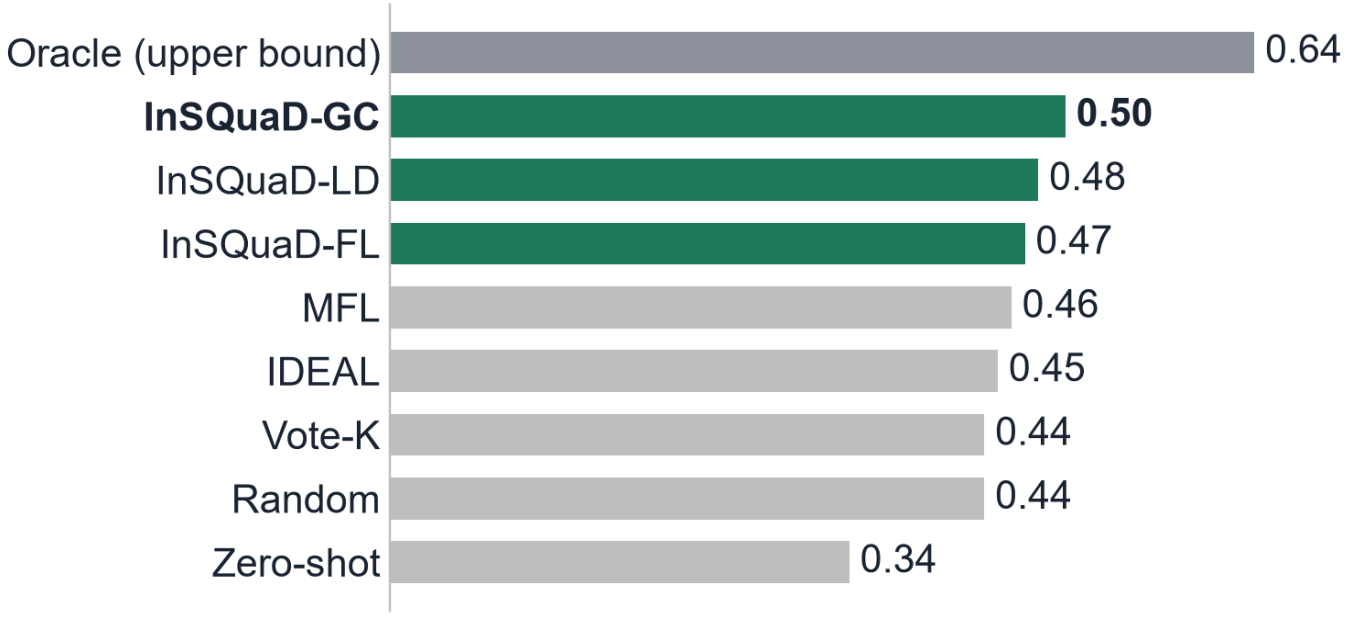
We used nine benchmarks:

- Classification: MRPC, SST5, MNLI, DBpedia, RTE.
- Multi-choice: HellaSwag.
- Dialogue: MWoZ.
- Generation: GeoQ, Xsum.

The LLMs are Gemma 2B (the main one), Gemma 7B and Davinci-002. The baselines are Zero-shot, Random, Diversity, Least Confidence, MFL, GC, Vote-K and IDEAL. Oracle is an upper bound. The main table uses Gemma 2B, an annotation budget of B = 18, and the mean of three trials.

Results

The next figure shows the average score over the nine benchmarks.

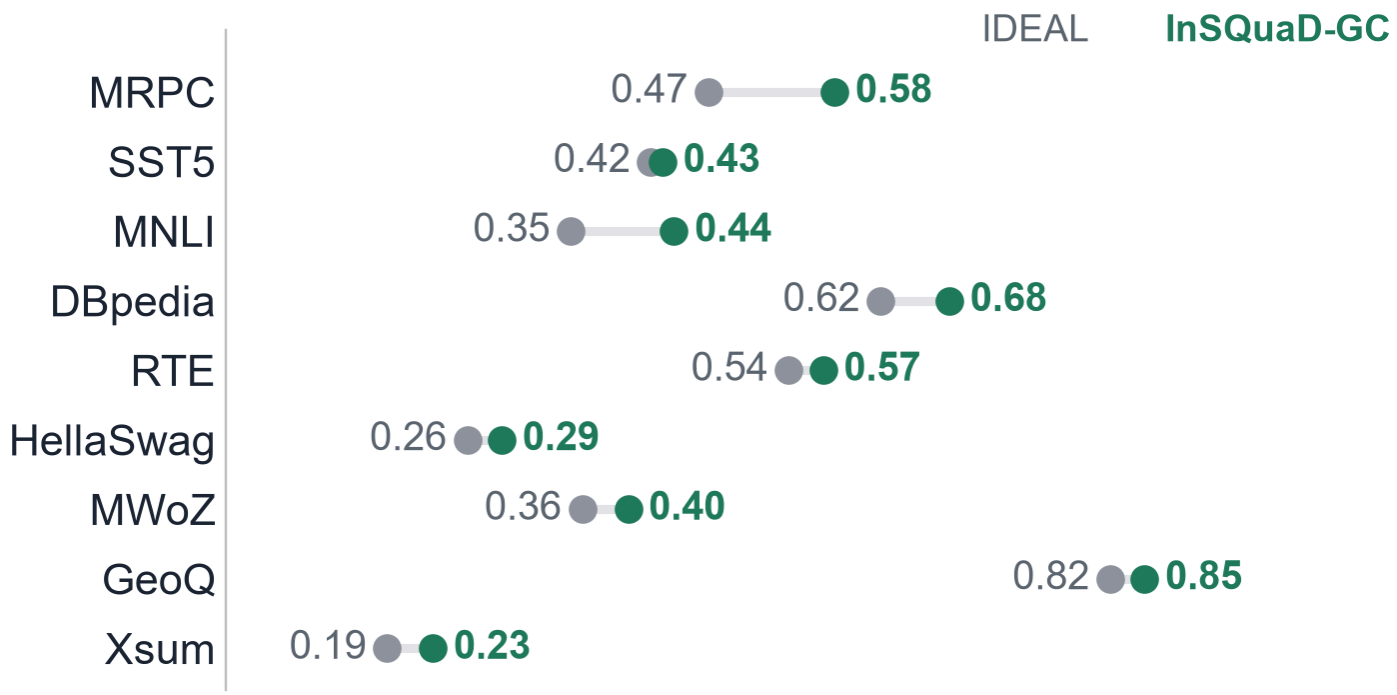


The averages are: Zero-shot 0.34, Random 0.44, Vote-K 0.44, IDEAL 0.45, MFL 0.46, InSQuaD-FL 0.47, InSQuaD-LD 0.48, InSQuaD-GC 0.50 and Oracle 0.64. All three InSQuaD variants are above all eight baselines on average score.

For average rank (lower is better), InSQuaD-GC has 3.6, InSQuaD-FL 5.7 and InSQuaD-LD 6.1. The best baseline, MFL, has 7.6.

The paper reports gains of up to 21.6% on classification, 16.4% on multi-choice and 7% on generation over existing baselines.

Here is InSQuaD-GC against IDEAL on each task.



InSQuaD-GC has a higher mean on all nine tasks. For example, MRPC goes from 0.47 to 0.58 and MNLI from 0.35 to 0.44. The SST5 gap is 0.01 (0.42 to 0.43), and that is inside the spread of the trials.

Trained against not trained

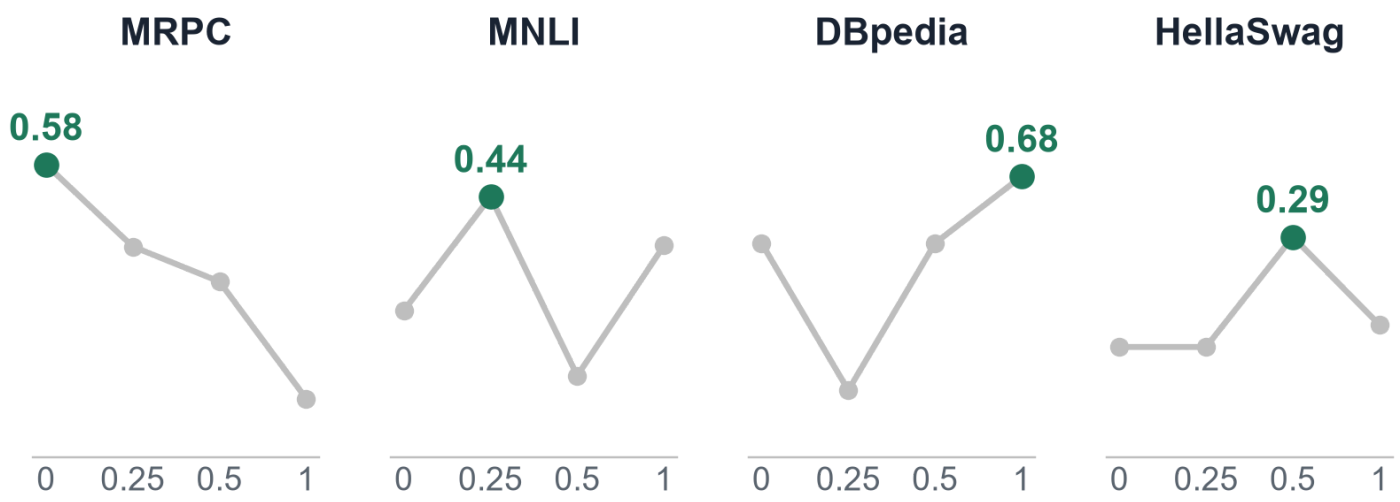
Is the gain from the selection rule or from the trained retriever? We ran the same selection rule with the pretrained retriever.

Variant	Average score, not trained	Average score, trained	Average rank, not trained	Average rank, trained
FL	0.43	0.47	10.3	5.7
LD	0.46	0.48	8.7	6.1
GC	0.46	0.50	8.4	3.6

With no training, the selection rule alone is level with the best baselines (0.43 to 0.46 against 0.46). The trained retriever adds the margin.

The quality-diversity weight

The weight lambda sets the balance between the quality loss and the diversity loss. We tested lambda = 0, 0.25, 0.5 and 1 for InSQuaD-GC.



Each panel has its own vertical scale. These are the numbers from Table IV of the paper.

Task	lambda = 0	lambda = 0.25	lambda = 0.5	lambda = 1
MRPC	0.58	0.51	0.48	0.38
MNLI	0.37	0.44	0.33	0.41
DBpedia	0.63	0.52	0.63	0.68
HellaSwag	0.24	0.24	0.29	0.25

No one value is best. The paper reports that lambda > 0 gives the best result on 7 of 9 tasks.

Other ablations

The paper gives these results in figures. I state the trends only.

- The method improves Gemma 2B, Gemma 7B and Davinci-002.
- InSQuaD-RETRIEVE is better than random exemplars and better than the top-k most similar. It has the highest gain on 7 of 9 tasks.
- A budget of 100 is not reliably better than a budget of 18.
- Greedy selection is much faster than the iterative Vote-K and IDEAL. Graph-Cut is the fastest variant and Log-Determinant the slowest.

Limitations

- We used only HotpotQA to build the training set.
- lambda needs calibration for each task.
- A gap to the Oracle remains: 0.50 against 0.64.

- The paper also names selection bias in annotation and retrieval, and interpretability, as future work.

Links

Slides: InSQuaD slides | Paper: arXiv 2508.21003 | Code: GitHub

What do you think?

0 Responses

Upvote Funny Love Surprised Angry Sad

0 Comments Login

Start the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS

Name

Share Best Newest Oldest

All articles

Causal-Consistency Training: An Idea From 2021 and the Research Since
10 Nov 2026

InSQuaD: Picking In-Context Examples That Are Relevant and Diverse
10 Nov 2026

The Squiggly Line Conjecture: An Idea From 2022 and What the Test Found
10 Nov 2026

Appeasing the Machine Spirit: RLHF in In-Context Learning

03 Nov 2026

Four Tricks at the Edge of Formal Language Theory
15 Aug 2026

Spring-Loaded Dinosaurs: What Penguins, Kiwis, and Armored Core Get Right (and Wrong) About Legs
28 Feb 2026

Passive Velocity Sorting: A Geometric Maxwell's Demon?
14 Dec 2025

AC Gravity and Gravitational Opacity
05 Dec 2025

Theorem: Equivalence of -arccoth and tan Functions
21 Nov 2025

The Photonic Transformer Hypothesis: Rethinking Photosynthesis and Methane Consumption in Plants
19 Nov 2025

The Scalable Quasi-Perpetual Photonic Machine
17 Nov 2025

How to Make a Ball Orbit Itself
08 Nov 2025

The Boron-Nitrogen Catastrophe: A Critical Gap in Beta Decay Verification
28 Oct 2025

X-Ray Data Pipeline: Ultra-High-Speed Communication Through Limestone Tubes
26 Oct 2025

The Universal Approximation Theorem Is Right. You're Using It Wrong.
19 Oct 2025

The Power Law Illusion: A Measurement Artifact Hypothesis
11 Oct 2025

Fractional Gamma Function via Fractional Derivatives
18 Sep 2025

On the cruel irony of the P-NP problem
18 Dec 2023

The Journey from India to Germany: A Guide for IT Professionals
20 Jun 2023

How does AutoGPT work under the hood?
03 May 2023

Unit Test Recorder - Automatically generate unit tests as you use your application
25 Jun 2020

How to migrate from vanilla Kubernetes to Istio service mesh?
14 Oct 2019